ISE 3434: Applied Optimization

Assignment 3

Due Date: October 20, 2026, 11:59pm

Directions

- Submit one PDF file (your write-up) and one Jupyter notebook (.ipynb) file to Canvas by the due date above. Late submissions are accepted according to the late policy in the syllabus.
- Answer every part of each problem in your PDF. Hand-drawn figures, such as graphs and branch and bound trees, are acceptable if they are scanned clearly and are easy to read.
- Your notebook should contain working code that prints your results, for every problem in which you use code.
- Grading considers both the correctness and clarity of your written solutions, and the correctness and readability of your code.

Problem 1: Branch and Bound (35 points)

A small furniture workshop is deciding how many tables to build today, and whether to rent an extra machine for the day. Each table earns a profit of $700, and renting the machine costs $500. Each table needs 2 hours of machine time. The workshop's own machine is available for 7 hours, and the rented machine would add 5 more hours. Each table also needs 2 hours of labor, installing the rented machine takes 1 hour of labor, and 10 hours of labor are available.

Let x be the number of tables built, and let y = 1 if the machine is rented and 0 otherwise. Working in hundreds of dollars, the problem is:

$$\begin{aligned} \max \quad & z = 7x - 5y \\ \text{s.t.} \quad & 2x - 5y \le 7 \qquad \text{(machine time)} \\ & 2x + y \le 10 \qquad \text{(labor)} \\ & x \ge 0 \text{ and integer}, \quad y \text{ binary} \end{aligned}$$

(a) Solve the LP relaxation graphically. In the LP relaxation, x is a continuous variable with $x \ge 0$, and y is a continuous variable with $0 \le y \le 1$. Draw the feasible region, list its corner points, and report the optimal solution and objective value.

(b) Round the LP solution from part (a) to the nearest integer values, and separately round both values down. Is either rounded point feasible for the original problem? What does this tell you about solving an integer program by rounding its LP relaxation?

(c) Solve the problem with branch and bound. So that everyone builds the same tree, use the following rules:

- Use LIFO (backtracking).
- If x is fractional, branch on x. Otherwise, branch on y.
- After branching, solve the child with $x \ge$ (for a branch on x) or $y = 1$ (for a branch on y) first.

You may solve each subproblem graphically or with Gurobi as a linear program (relax the integrality restrictions and add the branching constraints for that node). Either way, draw the tree and record, for each node, the order in which it was solved, its LP solution and objective value, and whether it was branched on or fathomed, and why.

(d) Branching on y and branching on x both split a subproblem into two new subproblems. In a few sentences, explain how the two kinds of branch differ.

To submit

- Your graph of the LP relaxation, its corner points, and its optimal solution (part a).
- Your answers to parts (b) and (d).
- Your branch and bound tree with the information requested in part (c), and the optimal solution and objective value of the problem.
- If you used Gurobi to solve the subproblems, your code as part of the single .ipynb file.

Problem 2: Genetic Algorithm for Cell Tower Placement (40 points)

In Problem 1 of Assignment 2, you solved the cell tower placement problem exactly and found that 7 towers are enough to cover all 25 regions. In this problem, you will solve the same problem with a genetic algorithm, using the same file city_adjacency.csv.

A solution is represented as a chromosome of 25 genes, one per region: a gene equals 1 if a tower is built in that region and 0 otherwise. Most chromosomes leave some region uncovered, so instead of rejecting them, the genetic algorithm minimizes a cost that penalizes uncovered regions:

$$\text{cost} = (\text{number of towers}) + \text{penalty} \times (\text{number of uncovered regions})$$

In lecture, infeasible chromosomes received a fitness of 0. Here, the penalty instead grows with the number of uncovered regions. Almost every random chromosome leaves some region uncovered (fewer than 3% of the random chromosomes in the first generation cover every region), so if all of them received the same worst score, the algorithm could not tell a chromosome that misses one region from one that misses ten.

The starter code **ga_cell_towers.ipynb** implements most of the genetic algorithm, following the steps from lecture with these specific choices:

- **Initial population:** 30 random chromosomes, in which each gene is 1 with probability 0.3.
- **Selection (tournament of 2):** to choose each parent, pick two chromosomes from the population at random and keep the one with the lower cost. This is a simple way to make fitter chromosomes more likely to become parents: a good chromosome wins most of the tournaments it enters, while a poor one rarely does.
- **Crossover:** one-point crossover at a random cut point, as in lecture.
- **Mutation:** each gene of a child is flipped independently with a small probability, called the mutation rate.
- **Keeping the best:** the best chromosome found so far is copied unchanged into each new generation, so it is never lost. The rest of the new generation consists of children.
- **Stopping rule:** 100 generations.

Use a penalty of 5 throughout. Unless a part says otherwise, use the default settings in the starter code: a population of 30, 100 generations, and a mutation rate of 1/25.

(a) Complete the two functions in Part 1 of the starter code: cost, which computes the cost above, and mutate, which flips each gene of a chromosome with a given probability. Run the test cell below them and include its output in your PDF.

(b) Run the genetic algorithm with a penalty of 5 for seeds 0 through 9, and report the number of towers and the number of uncovered regions in the best solution of each run. How many runs found an optimal solution? Could the genetic algorithm tell, on its own, that it had found an optimal solution? Explain. Also include the plot of the best cost over the generations for seed 0.

(c) Repeat part (b) with a mutation rate of 0. Compare the results with part (b), and explain what role mutation plays in a genetic algorithm.

To submit

- Your answers to parts (a) through (c), including the test output, the tables of results, and the plot.
- Your completed starter code as part of the single .ipynb file, with all outputs shown.

Problem 3: Designing a Heuristic (25 points)

In Problem 4 of Assignment 1, you used Gurobi to find that 3 guards are enough to cover the 8 overnight shifts of a town festival. Recall the problem: every shift must be covered by exactly one guard, and a guard cannot be assigned to two shifts that conflict with one another. The conflicts are listed below (the relationship goes both ways).

Shift	Conflicts With
1	2, 3
2	1, 3, 4
3	1, 2, 5
4	2, 5, 6
5	3, 4, 6, 7
6	4, 5, 7
7	5, 6, 8
8	7

In this problem, you will design your own heuristic for assigning shifts to guards. Do not use Gurobi or any code.

(a) Describe a heuristic for assigning the shifts to guards, as a list of numbered steps that anyone could follow without further explanation. Explain why your heuristic always produces a feasible schedule, that is, one in which every shift has exactly one guard and no guard works two conflicting shifts.

(b) Apply your heuristic by hand to the 8 shifts above. Show each step, and report the number of guards and the shifts assigned to each guard.

(c) Shifts 1, 2, and 3 all conflict with one another. Use this fact to explain why no schedule can use fewer than 3 guards. Is your schedule optimal? Then describe one situation in which your heuristic could perform poorly, or one change that could improve it.

To submit

- Your answers to parts (a) through (c), in your PDF. No code is required for this problem.